

RP 서버 API 명세서 — 클라이언트(앱/웹) 개발자용

앱·웹 클라이언트가 **RP 서버**에 호출하는 API 규격이다. 패스키 회원가입·로그인·자격증명 관리를 클라이언트에서 구현하기 위해 필요한 모든 엔드포인트를 담는다.

🔍 빠른 조회만 필요하다면 → [rp-client-api-quickref.md](#) (Swagger 스타일 엔드포인트·스키마 요약).
이 문서는 흐름·배경·구현 가이드를 포함한 상세본이다.

이 문서가 다루는 범위

- 클라이언트 ↔ **RP 서버** 통신만. 클라이언트는 Passkey 플랫폼 서버를 직접 호출하지 않는다.
- 엔드포인트 경로·요청/응답은 RP 서버 기준이다. ceremony 5종(`/passkey/**`)은 Java RP SDK가 고정 제공하고, 회원·자격증명 엔드포인트(`/users` , `/me` , `/credentials`)는 RP가 자체 구현하는 영역이라 **RP별로 경로가 다를 수 있다** — 아래는 참조 RP 서버(`passkey-rp-demo`) 기준이다.
- 응답 형식은 **RP 서버 전체가 Passkey 플랫폼의 `ApiResponse` envelope를 따른다** — ceremony든 RP 자체 엔드포인트든 동일 스키마. 자세한 내용은 아래 “응답 Envelope” 참조.
- API Key·플랫폼 시크릿은 RP 서버 내부에만 있다. 클라이언트는 어떤 시크릿도 다루지 않는다.

0. 공통 규약

BASE URL

RP 서버 주소. 예: `https://rp.example.com` (데모: `http://localhost:8090`)

응답 ENVELOPE

RP 서버의 모든 응답(ceremony·회원·자격증명 엔드포인트 전부)은 아래 envelope로 감싸진다. Passkey 플랫폼 서버의 `ApiResponse<T>` 템플릿과 동일한 스키마라, 클라이언트는 RP 서버든 플랫폼이든 한 가지 형태만 파싱하면 된다. 실제 데이터는 `data` 필드에 있다.

```
{
  "success": true,
  "code": "OK",
  "message": "Success",
  "data": { ... },
  "traceId": "a1b2c3d4...",
  "timestamp": "2026-05-22T10:00:00"
}
```

실패 시 (`data` 는 생략, `error` 가 채워짐):

```
{
  "success": false,
  "code": "P003",
  "message": "Challenge expired or not found",
  "error": { "errorCode": "P003", "fieldErrors": null },
  "traceId": "a1b2c3d4...",
  "timestamp": "2026-05-22T10:00:00"
}
```

필드	설명
<code>success</code>	성공 여부 (boolean)
<code>code</code>	성공 시 "OK", 실패 시 에러 코드
<code>message</code>	사람이 읽는 메시지
<code>data</code>	실제 응답 데이터 — 성공 시에만 (실패 시 생략)
<code>error</code>	{ <code>errorCode</code> , <code>fieldErrors</code> } — 실패 시에만 (성공 시 생략)
<code>traceId</code>	추적 ID — 장애 문의 시 이 값을 전달
<code>timestamp</code>	응답 생성 시각

클라이언트는 항상 `success` 로 분기하고, 성공이면 `data` 를, 실패면 `code` / `error.errorCode` 를 읽으면 된다 — 엔드포인트별로 파싱을 달리할 필요가 없다.

인증

구분	인증 방식
ceremony 엔드포인트 (<code>/passkey/register/*</code> , <code>/passkey/authenticate/*</code>)	불필요 — 누구나 호출. ceremony 자체가 본인 확인
<code>/passkey/refresh</code>	불필요 (refreshToken 본문으로 전달)
보호 API (<code>/me</code> , <code>/credentials</code> 등)	필요 — <code>Authorization: Bearer <accessToken></code> 헤더

`accessToken` 은 로그인(`authenticate/finish`) 성공 시 발급된다. 만료(기본 15분) 시 `/passkey/refresh` 로 갱신한다.

바이너리 인코딩

WebAuthn은 바이너리 데이터를 다룬다. 요청 본문의 `*B64u` 필드는 모두 **Base64URL**(`+/=` 없는 변형) 인코딩 문자열이다. `navigator.credentials` 결과의 `ArrayBuffer`를 Base64URL로 변환해 전송한다.

1. 패스키 등록 (회원가입 + 패스키 생성)

흐름

- ① POST `/users` → `externalUserId` 획득 (RP 회원 생성)
- ② POST `/passkey/register/begin` → 등록 옵션(challenge 등) 획득
- ③ `navigator.credentials.create()` → 브라우저가 패스키 생성
- ④ POST `/passkey/register/finish` → 생성 결과 전송, 서버 검증·저장

① 회원 생성 — POST `/USERS`

RP가 자체 구현하는 회원가입. 패스키 등록 전에 `externalUserId`를 확보한다.

요청

```
{ "displayName": "홍길동" }
```

응답 (data 안)

```
{ "externalUserId": "user-3f9a...", "displayName": "홍길동" }
```

`externalUserId`는 RP가 사용자에게 부여하는 안정적 식별자다. 이후 모든 ceremony에서 이 값을 쓴다. 플랫폼 서버는 `displayName`을 보지 않는다(RP 소유 정보).

② 등록 시작 — POST `/PASSKEY/REGISTER/BEGIN`

요청

```
{ "externalUserId": "user-3f9a...", "displayName": "홍길동" }
```

응답 (data 안)

```
{
  "ceremonyId": "310d2d97-...",
  "challenge": "gTom0uXJ...",
  "rp": { "id": "example.com", "name": "Example" },
  "user": { "id": "qbm12K0d...", "name": "user-3f9a...", "displayName": "홍길동" },
  "pubKeyCredParams": [
    { "type": "public-key", "alg": -7 },
    { "type": "public-key", "alg": -257 }
  ],
  "timeout": 60000,
  "attestation": "none",
  "authenticatorSelection": {
    "userVerification": "preferred",
    "residentKey": "preferred",
    "requireResidentKey": false
  },
  "excludeCredentials": [
    { "type": "public-key", "id": "b51Pb6...", "transports": "internal" }
  ]
}
```

`ceremonyId` 는 ④에서 다시 보내야 하므로 보관한다.

③ 브라우저에서 패스키 생성

②의 응답을 `PublicKeyCredentialCreationOptions` 로 변환해 호출. `challenge` · `user.id` · `excludeCredentials[].id` 는 Base64URL → `ArrayBuffer` 로 디코딩해야 한다.

```
const cred = await navigator.credentials.create({ publicKey: { /* ②의 값 매핑 */ } });
```

반드시 사용자 제스처(버튼 클릭 등) 안에서 호출해야 브라우저가 허용한다.

④ 등록 완료 — `POST /PASSKEY/REGISTER/FINISH`

요청

```
{
  "ceremonyId": "310d2d97-...",
  "credentialId": "<cred.id>",
  "clientDataJsonB64u": "<cred.response.clientDataJSON, Base64URL>",
  "attestationObjectB64u": "<cred.response.attestationObject, Base64URL>",
  "transports": "internal,hybrid",
  "nickname": "내 iPhone"
}
```

`nickname`은 사용자가 기기를 알아보게 하는 라벨(선택). `transports`는 `cred.response.getTransports()` 결과를 콤마로 join.

응답 (`data` 안)

```
{ "credentialDbId": "32e31519-...", "credentialId": "b51Pb6...", "aaguid":
  "fbfc3007-..." }
```

2. 패스키 로그인

흐름

- ① POST /passkey/authenticate/begin → 인증 옵션 획득
- ② navigator.credentials.get() → 브라우저가 패스키로 서명
- ③ POST /passkey/authenticate/finish → 서명 전송, 검증 후 토큰 발급

① 인증 시작 — POST /PASSKEY/AUTHENTICATE/BEGIN

요청

```
{ "externalUserId": "user-3f9a..." }
```

응답 (`data` 안)

```
{
  "ceremonyId": "a3462d77-...",
  "challenge": "kPx9...",
  "timeout": 60000,
  "rpId": "example.com",
  "allowCredentials": [
    { "type": "public-key", "id": "b51Pb6...", "transports": "internal" }
  ],
  "userVerification": "preferred"
}
```

② 브라우저에서 서명

`challenge` · `allowCredentials[0].id`를 Base64URL → `ArrayBuffer`로 디코딩 후 호출.

```
const assertion = await navigator.credentials.get({ publicKey: { /* ①의 값 매핑 */
} }));
```

③ 인증 완료 — `POST /PASSKEY/AUTHENTICATE/FINISH`

요청

```
{
  "ceremonyId": "a3462d77-...",
  "credentialId": "<assertion.id>",
  "clientDataJsonB64u": "<assertion.response.clientDataJSON, Base64URL>",
  "authenticatorDataB64u": "<assertion.response.authenticatorData, Base64URL>",
  "signatureB64u": "<assertion.response.signature, Base64URL>",
  "userHandleB64u": "<assertion.response.userHandle, Base64URL>"
}
```

응답 (`data` 안)

```
{
  "credentialDbId": "32e31519-...",
  "tenantUserId": "3dfb2e44-...",
  "credentialId": "b51Pb6...",
  "signatureCounter": 0,
  "accessToken": "eyJhbGciOiJSUzI1Ni...",
  "refreshToken": "eyJhbGciOiJSUzI1Ni...",
  "accessExpiresIn": 900
}
```

`accessToken` 을 이후 보호 API 호출에 `Authorization: Bearer` 로 사용한다. `accessExpiresIn` 은 액세스 토큰 유효 시간(초).

RP 서버 설정에 따라 인증 성공 시 서버가 **세션 쿠키**를 내려줄 수도 있다(SESSION 모드). 그 경우 토큰을 직접 저장하지 않고 쿠키로 인증한다 — RP 서버팀과 확인.

3. 토큰 갱신 — `POST /passkey/refresh`

`accessToken` 만료 시 호출. `refreshToken` 은 1회용이라 호출할 때마다 새 값으로 교체된다.

요청

```
{ "refreshToken": "eyJhbGciOiJSUzI1Ni..." }
```

응답 (`data` 안)

```
{
  "accessToken": "eyJhbGciOiJSUzI1Ni... (새 값)",
  "refreshToken": "eyJhbGciOiJSUzI1Ni... (새 값)",
  "accessExpiresIn": 900
}
```

⚠️ 응답의 새 `refreshToken` 으로 반드시 교체 저장할 것. 이전 `refreshToken` 을 재사용하면 탈취로 간주되어 토큰 패밀리 전체가 무효화된다(재로그인 필요).

4. 보호 API — 로그인한 사용자 전용

모든 요청에 `Authorization: Bearer <accessToken>` 헤더 필요. 누락·만료·위조 시 **401 + envelope** 에러(`code: "D001"`), 응답 헤더 `WWW-Authenticate: Bearer error="token_expired"|"invalid_token"`.

아래 경로·응답은 참조 RP 서버(`passkey-rp-demo`) 기준이다. 실제 RP 서버는 경로·필드를 다르게 구현할 수 있다. 응답은 §0의 `ApiResponse` envelope를 따른다 — 아래 예시는 모두 envelope의 `data` 필드 내용이다.

4.1 내 정보 조회 — `GET /ME`

응답 (`data` 안)

```
{ "externalUserId": "user-3f9a...", "displayName": "홍길동" }
```

사용자 미존재 시 envelope 에러 `code: "D002"` (HTTP 404).

4.2 내 패스키 목록 — GET /CREDENTIALS

응답 (data 안 — 배열)

```
[
  {
    "id": "32e31519-...",
    "credentialId": "b51Pb6...",
    "nickname": "내 iPhone",
    "status": "ACTIVE",
    "aaguid": "fbfc3007-...",
    "transports": "internal,hybrid",
    "signatureCounter": 3,
    "lastUsedAt": "2026-05-22T09:00:00Z",
    "createdAt": "2026-05-20T12:00:00Z",
    "revokedAt": null,
    "revokedReason": null
  }
]
```

4.3 패스키 이름 변경 — PATCH /CREDENTIALS/{ID}

{id} 는 4.2 응답의 id (UUID).

요청

```
{ "nickname": "갤럭시 S24" }
```

응답 (data 안): 변경된 credential 객체 (4.2 항목과 동일 형태)

4.4 패스키 삭제 — DELETE /CREDENTIALS/{ID}

응답: HTTP 200 + envelope (success: true, data 없음)

```
{ "success": true, "code": "OK", "message": "Success", "traceId": "...", "time-
stamp": "..." }
```


보호 API는 `accessToken` 에서 추출한 사용자 본인의 자격증명만 다룬다. 다른 사용자의 `credentialId` 를 알아도 조작 불가(서버가 소유권 검증).

5. 에러 코드

응답이 `success: false` 이면 `code` (= `error.errorCode`)로 분기한다. 코드는 도메인 prefix 1글자 + 3자리 숫자다 — `C/A/T/P/R/M` 은 Passkey 플랫폼 도메인, `D` 는 RP 서버 자체 도메인.

클라이언트가 자주 만나는 코드:

코드	의미	클라이언트 대응
<code>C001</code>	입력값 검증 실패	요청 본문 필드 확인 (<code>error.fieldErrors</code> 참조)
<code>P001</code>	테넌트 WebAuthn 설정 없음	RP 서버/플랫폼 설정 문제 — 운영팀 문의
<code>P003</code>	challenge 만료/없음	ceremony를 처음부터 다시 (begin부터)
<code>P004</code>	서명 검증 실패	패스키 불일치 — 다른 패스키로 재시도
<code>P005</code>	signature counter 이상	보안 경고 — 해당 패스키 사용 중단 권고
<code>P006</code>	자격증명 소유권 불일치	본인 자격증명만 관리 가능
<code>D001</code>	인증 필요 (RP 서버 — 토큰 누락/무효)	<code>/passkey/refresh</code> 로 갱신, 실패 시 재로그인
<code>D002</code>	사용자 없음 (RP 서버)	회원 가입 흐름 재진입
<code>429</code> *	호출 한도 초과 (rate limit)	잠시 후 재시도

* `429` 는 HTTP status — rate limit은 envelope 이전 단계에서 차단될 수 있어 `code` 대신 status로 판단.

전체 에러 코드는 [error-codes.md](#) 참조 (`D` -prefix RP 서버 코드는 RP 구현마다 다를 수 있음).

6. 클라이언트 구현 체크리스트

- ☐ `navigator.credentials.create/get` 은 사용자 제스처 안에서 호출
- ☐ `challenge` · `*.id` · `userHandle` 등 바이너리는 **Base64URL** 인코딩/디코딩
- ☐ `ceremonyId` 를 begin → finish 사이에 보관

- ☐ 모든 응답은 `ApiResponse` `envelope` — `success` 로 분기 후 성공이면 `data` , 실패면 `code` 를 읽음
- ☐ `accessToken` 은 보호 API 호출에 `Authorization: Bearer` 로 첨부
- ☐ `refreshToken` 은 갱신할 때마다 **새 값으로 교체 저장**
- ☐ WebAuthn 미지원 브라우저 감지 (`window.PublicKeyCredential` 존재 확인)
- ☐ 사용자 취소(`NotAllowedError`)와 실제 오류를 구분해 안내
- ☐ 401 응답 시 `refresh` → 실패하면 로그인 화면으로

브라우저 웹이라면 `@crosscert/passkey-sdk` (JS SDK)가 위 ceremony 인코딩을 래핑한다 — [integration-guide.md](#) 참조. 네이티브 앱은 OS 패스키 API(iOS `ASAuthorization` , Android `Credential Manager`)로 ②③ 단계를 대체하고, begin/finish HTTP 호출은 동일하다.